MySQL reservation tables

Every table and column used to send a quote, charge a card, or make a reservation, with its datatype and keys. Anything empty or dead is left out.

The join key is `SOLICITUD`. Never join on `boleta`.

```
bita2          SOLICITUD (PK)     quote / request + payment
  ├─ bita_reserva.SOLICITUD       confirmed reservation
  ├─ resecut.bitacora             points ledger
  ├─ allotment.bitacora           unit consumed
  ├─ comen.bitacora               notes + charge log
  └─ saldofav.bitacora            credit balance movement
```

None of these joins is a declared foreign key. The schema has no FK constraints on any reservation table — every relationship above is convention only. `resecut.bitacora` and `allotment.bitacora` are not even indexed.

Which tables each flow touches

FLOW	TABLES
send a quote	**bita2** · **canual** · **condir** · **tarifa_hotel3** · **tasa** · **destino** · **tipo_unid** · **combinaciones** · **temp_combinaciones** · **allotment** · **allotnoc** · **allotnocb** · **solicitudes_allotment** · **detalle_soli_allotment** · **pasante** · **bono_otorgado** · **junto_pendiente_actualizar** · **emergente**
charge a card	**credit_cards** · **credit_cards_log** · **issuing_bank** · **bin_list_tarjetas** · **bin_list_tarjetas_gua** · **bin_cache** · **network** · **type_acquiring** · **tipo_tarjeta** · **forma_pago** · **saldofav** · **comen**
make a reservation	**bita_reserva** · **bita_reserva_audit** · **resecut** · **allotment** · **destino** · **distribucion** · **vencimiento_detalle** · **saldofav** · **bono_otorgado** · **boletas_eliminadas** · **comen** · **sec_4users**

The write path is stored procedures

Nothing writes these tables directly. The form calls stored procedures, and there are 49 of them in the schema. Reimplementing the flows in Postgres means reimplementing these — a table-level copy will not reproduce the behaviour.

PROCEDURE	WRITES	READS
crearBitacora	**bita2**	canual condir pasante bono_otorgado tasa
CERRAR_BITACORAS	**bita2**	
llenar_temp_combinaciones	**temp_combinaciones**	allotment tipo_unid
crearReserva	**bita_reserva saldofav**	bita2 allotment sec_4users
ELIMINAR_RESERVA	**allotment bita_reserva comen resecut saldofav**	pasante
CANCELACION_RESERVA_CON_SALDO_A_FAVOR	**allotment bita_reserva comen resecut saldofav**	bita2 pasante
RESERVA_CON_SALDO_A_FAVOR	**comen saldofav**	bita2 bita_reserva pasante
PASAR_DE_R_A_U_RESERVAS	**bita2 bita_reserva destino pasante resecut**	
RESERVA_USANDO_PUNTOS_LIBERADOS_AI	**resecut vencimiento_detalle**	
RECORRER_RESERVA / USODEPUNTOS2	**distribucion**	pasante resecut
crear_tarjeta_credito	**credit_cards issuing_bank**	bin_list_tarjetas
actualizar_tarjetas_credito		credit_cards pasante
actualizarCuenta	**pasante**	canual resecut saldofav bono_otorgado
allotment_ano / allotment_mes	**allotnocg allotnocr**	allotnoc allotnocb

`crear_tarjeta_credito` does `INSERT IGNORE INTO issuing_bank` before inserting the card, so the issuer list grows itself from whatever the agent types.

There are **no triggers** in this schema. Anything that looks trigger-maintained, including `bita_reserva_audit`, is written by application code or one of these procedures.

Contents

comen — notes and charges

pasante — member

junto_pendiente_actualizar

emergente

allotment — inventory

solicitudes_allotment — inventory request

bono_otorgado

saldofav — credit balance

credit_cards — cards on file

credit_cards_log

Card lookups

Lookup tables

Coded values

Legacy card columns

bita2

One row per quote, booked or not. `SOLICITUD` is the Bitácora No the agent sees — MySQL assigns it via `auto_increment`, the form does not supply it. Next value 325096, and the range 122503–325095 across 92,823 rows is sparse.

IDENTITY AND AUDIT

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**SOLICITUD**	int	PK, auto_inc	the Bitácora No
bita2	**JUNTO**	varchar(7)	index	membership number
bita2	**FECHA**	date	index	date quoted
bita2	**HORA**	varchar(8)		`HH:MM:SS` on all rows
bita2	**USUARIO**	varchar(20)	index	agent who opened it, 44 distinct
bita2	**USUARIOULT**	varchar(20)		agent who last touched it
bita2	**FECHA_ACTU**	timestamp		last update
bita2	**REFIERE**	varchar(8)		referral

OUTCOME AND CHANNEL

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**RESULTADO**	varchar(16)	index	the booked flag

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**ESTATUS**	varchar(9)		workflow state, independent of `RESULTADO`
bita2	**TIPOSOL**	varchar(15)		the channel, labelled Medio on the form; values from `tipo_sol`
bita2	**ASUNTO**	varchar(15)		labelled Tipo Reserva; values from `tipo_reserva`
bita2	**SOLUCION**	varchar(10)		`SOLIRES` on every completed quote

`RESULTADO = 'confirmada'` is what actually means booked: those rows have a `bita_reserva` row almost always, `Informacion` rows almost never, `No hay espacios` and `Lista de Espera` never. It is independent of `ESTATUS` — `confirmada` + `pendiente` occurs on about 750 rows.

CONTACT AND BILLING IDENTITY

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**NOMBRERES**	varchar(40)		guest name
bita2	**TELEFONO**	varchar(12)		phone
bita2	**FAX**	varchar(11)		not a fax — a second phone
bita2	**MAIL**	varchar(45)		email
bita2	**NIT**	varchar(10)		tax id, not numeric
bita2	**NITNOMBRE**	varchar(40)		billing name
bita2	**HABLADO**	varchar(40)		who was spoken to

ROOM BLOCKS

A quote carries up to three room blocks in parallel column families, distinguished by a numeric suffix. Block 1 is on every quote, block 2 on about a tenth, block 3 rarely. All of these live in `bita2`.

TABLE	ROLE	BLOCK 1	BLOCK 2	BLOCK 3	TYPE
bita2	hotel	HOTEL1	–	–	`int`, indexed
bita2	check-in	FECHADEL1	**FECHADEL2**	FECHADEL3	`date`
bita2	check-out	FECHASAL1	**FECHAAL2**	FECHAAL3	`date`
bita2	adults	ADULTOS	**ADULTOS2**	ADULTOS3	`varchar(2)`
bita2	children	NINOS	**NINOS2**	NINOS3	`varchar(2)`, `NINOS3` is `varchar(1)`

TABLE	ROLE	BLOCK 1	BLOCK 2	BLOCK 3	TYPE
bita2	room code	HABITA1	HABITA2	HABITA3	varchar(20), HABITA3 is varchar(19)
bita2	unit count	unidad1	unidad2	unidad3	int
bita2	season	destino1	destino2	destino3	varchar(10)
bita2	season alt	destino11	destino22	destino33	varchar(10)
bita2	nights	noches1	noches2	noches3	int
bita2	nights alt	noches11	noches22	noches33	int
bita2	points	punto	punto2	punto3	decimal(6,2)
bita2	maint. points	–	manto_pts2	manto_pts3	decimal(6,1)
bita2	maint. amount	–	manto_pts2_d	manto_pts3_d	decimal(12,2)
bita2	reservation type	TIPO_RESERVA	–	–	varchar(100)

Consider normalising these into a child table with one row per block rather than carrying 39 columns across.

Four traps in that family:

`HABITA1` holds the room code, not a description — `D2DL` `B2BR` `STL` `B1BR` `V1BR` `D2JSE` `PROS` `OTROS`. Same pre-split codes as `resecut.CVEUNI`, so `D2DL` needs resolving to `D2DLA` / `D2DLP` by hotel, exactly like the allotments backfill.

`unidad1` is the number of units (1–7), not a room id.

`ADULTOS` and `NINOS` are `varchar`, not ints, though every value parses as a number (max 80). An unused block stores `''` or `'0'`, never `NULL` — so treat blank as absent, or a block-2 row appears with 0 adults instead of not existing.

`TIPO_RESERVA2`, `TIPO_RESERVA3` and `TIPO_RESERVA4` exist and are nearly always populated, but each is a copy of `TIPO_RESERVA` written on every row regardless of how many blocks the quote has. No per-block information. There is also a fourth block (`fechadel4`, `habita4`, `punto4`, `noches4`, `destino4`, `unidad4`, `manto_pts4`) that is entirely empty — what looks like data in `adultos4` and `ninos4` is the literal string `'0'`.

POINTS AND MONEY

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**PUNLOC**	decimal(6,1)		local points

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**PUNDIS**	decimal(13,1)		points available at quote time
bita2	**BONODIS**	decimal(8,1)		bonus available at quote time
bita2	**DIFPUNINT**	decimal(10,2)		points difference
bita2	**TOTAL**	decimal(10,2)		USD, −1188 to 11884 — negatives exist
bita2	**TOTALQ**	decimal(10,2)		quetzales, −6014.92 to 92814.04
bita2	**TCAMBIO**	decimal(10,2)		rate used; max 297.50 is garbage against a real rate near 7.62
bita2	**MANTOPUNTO**	decimal(10,2)		points maintenance fee
bita2	**CUOTAANUAL**	decimal(10,2)		annual fee
bita2	**CUOTAMEM**	decimal(10,2)		membership fee
bita2	**saldofav**	decimal(12,2)		credit balance applied
bita2	**FECHA_PAGO**	date		payment date, 224 zero dates

`TOTAL = 360.00` with `TCAMBIO = 7.62` gives `TOTALQ = 2743.20`, so the three are consistent where the rate is sane.

GUEST SURCHARGE AND PAYMENT REFERENCE

TABLE	COLUMN	TYPE	KEY	NOTE
bita2	**INVITADO**	tinyint(1)		0/1 flag
bita2	**TOTAL_INV**	decimal(14,2)		the amount, set on 2,220 of the 2,238 flagged rows
bita2	**CHEQUENO**	varchar(45)		cheque ref
bita2	**CHEQUEBCO**	varchar(20)		cheque bank
bita2	**DEPOSITONO**	varchar(10)		deposit ref
bita2	**DEPOSITOBC**	varchar(10)		deposit bank

The card columns on `bita2` are legacy duplicates of `credit_cards` — see Legacy card columns.

tarifa_hotel3

The points rate per date, season and unit. This is what prices a quote.

TABLE	COLUMN	TYPE	KEY	NOTE
tarifa_hotel3	**corr**	int	PK, auto_inc	
tarifa_hotel3	**FECHA**	date	index	the night being priced
tarifa_hotel3	**temporada**	enum		`ENTRE SEMANA` `FIN DE SEMANA` `SUPER ALTA` `PROMO` `ENTRE FIN`
tarifa_hotel3	**dia**	varchar(10)		day of week
tarifa_hotel3	**hotel**	int		hotel
tarifa_hotel3	**unidad**	int		unit type
tarifa_hotel3	**puntos**	int		**the rate**
tarifa_hotel3	**tipo_punto**	varchar(50)		points category
tarifa_hotel3	**estado**	tinyint(1)		active
tarifa_hotel3	**created_at**	datetime		
tarifa_hotel3	**updated_at**	datetime		auto-updates

Covers 20,127 date/unit rows out to 2027-12-31. `tarifa_hotel` and `tarifa_hotel2` are older generations — `tarifa_hotel2` stops at 2023.

tasa

The daily USD/GTQ rate, one row per day, currently to 2026-07-31. This is the source for `bita2.TCAMBIO` and is what `app/cron_jobs/sync_exchange_rate.py` maintains.

TABLE	COLUMN	TYPE	KEY	NOTE
tasa	**corr**	int	PK, auto_inc	
tasa	**fecha**	date	index	one row per day
tasa	**tasa**	decimal(6,2)		rate

bita_reserva

One row per booking. `boleta` is the number the member is told — also `auto_increment` (next 302833), also not supplied by the form.

TABLE	COLUMN	TYPE	KEY	NOTE
bita_reserva	**boleta**	int	PK, auto_inc	the member-facing number
bita_reserva	**SOLICITUD**	int	join → bita2, indexed	the join
bita_reserva	**JUNTO**	varchar(7)	index	membership number
bita_reserva	**FECHA**	date	index	booked on — use this, not `feccon`
bita_reserva	**FECHASOL**	date		date of the original quote
bita_reserva	**HOTEL1**	int	index	hotel
bita_reserva	**FECHADEL1**	date	index	check-in
bita_reserva	**FECHASAL1**	date		check-out
bita_reserva	**ADULTOS**	varchar(2)		text, not int
bita_reserva	**NINOS**	varchar(2)		text, not int
bita_reserva	**noches**	int		nights quoted
bita_reserva	**noches_real**	int		nights actually stayed
bita_reserva	**tipo_habita**	varchar(70)		free text (`1 BÚNGALO DE 4`), not a room code
bita_reserva	**habita_real**	text		room actually given
bita_reserva	**puntos_reserva**	decimal(7,1)		points charged
bita_reserva	**NOMBRERES**	varchar(40)		guest
bita_reserva	**TELEFONO**	varchar(12)		phone
bita_reserva	**MAIL**	varchar(45)		email
bita_reserva	**ESTATUS**	varchar(50)	index	`UTILIZADA` / `CANCELADO` — different vocabulary from `bita2.ESTATUS`
bita_reserva	**ASUNTO**	varchar(25)		reservation type
bita_reserva	**confirmahotel**	varchar(20)		hotel confirmation code
bita_reserva	**confirmas**	varchar(80)		second confirmation code
bita_reserva	**observa**	text		notes
bita_reserva	**USUARIO**	varchar(20)	index	agent
bita_reserva	**USUARIOULT**	varchar(8)		last agent

`feccon` is `0000-00-00` on all 51,080 rows and `HORA` is blank on every row — both look useful and hold nothing.

bita_reserva_audit

A trigger-written audit trail, one row per change, 51,128 rows and current to today.

TABLE	COLUMN	TYPE	KEY	NOTE
bita_reserva_audit	**id**	bigint	PK, auto_inc	
bita_reserva_audit	**boleta**	int	index → bita_reserva	which reservation
bita_reserva_audit	**changed_column**	varchar(64)		always `usuario`
bita_reserva_audit	**new_value**	text		the new value
bita_reserva_audit	**changed_by**	varchar(100)		MySQL user
bita_reserva_audit	**changed_at**	datetime	index	when
bita_reserva_audit	**change_type**	varchar(10)		operation
bita_reserva_audit	**connection_id**	int		MySQL connection

Worth knowing before you rely on it: `changed_column` is `usuario` on all 51,128 rows, so the trigger only tracks that one column. `old_value` is populated on 0.1% of rows and `extra_info` is empty, so you cannot reconstruct a before/after history from this table.

resecut

One to six rows per reservation. The only table that carries the room code on the booked side.

TABLE	COLUMN	TYPE	KEY	NOTE
resecut	**CORR**	int	PK, auto_inc	
resecut	**bitacora**	int	join → bita2, **not indexed**	the join, not named `SOLICITUD`
resecut	**BOLETA**	varchar(7)	index	reservation number, text
resecut	**JUNTO**	varchar(7)	index	membership number
resecut	**CVEUNI**	varchar(10)		room code, pre-split `D2DL`
resecut	**NUMPUN**	decimal(10,1)		points on this line
resecut	**cant_uni**	int		units of this room type
resecut	**noc_fin**	int		weekend nights
resecut	**noc_entre**	int		weekday nights

TABLE	COLUMN	TYPE	KEY	NOTE
resecut	**NUMSEM**	int		week number
resecut	**SEMNOC**	int		weeks/nights flag
resecut	**destino_res**	varchar(12)		season code, e.g. `20261023`
resecut	**FECDIA**	date	index	32 zero dates
resecut	**FECTRA**	date		transaction date
resecut	**PERIODO**	varchar(7)		period
resecut	**NUMCON**	int		contract
resecut	**NUMANO**	int		contract year
resecut	**SECPUN**	int	index	sequence
resecut	**FE**	varchar(2)		
resecut	**TIPDOC**	varchar(3)		document type
resecut	**nor_bono**	varchar(12)	index	points / bonus / certificate
resecut	**CUESTA**	varchar(2)	index	used / cancelled / returned
resecut	**TIPCAN**	varchar(2)		cancellation reason
resecut	**DESUSO**	varchar(20)		usage note
resecut	**desdoc**	varchar(60)		document description
resecut	**certi**	varchar(20)		certificate ref
resecut	**nconfir**	varchar(80)		confirmation number
resecut	**FACTURA**	varchar(20)		invoice
resecut	**LOCALINTER**	varchar(5)		local / international
resecut	**impcuo**	decimal(12,2)		amount due
resecut	**imppag**	decimal(12,2)		amount paid
resecut	**NOMBRE1**	varchar(40)		member name

`CVEUNI` carries the shared `D2DL`; our `room_types` splits it into `D2DLA` / `D2DLP`. Resolve by hotel, same as the allotments backfill.

boletas_eliminadas

Deleted reservations, kept out of `resecut` so the points ledger stays clean. 452 rows, most recent 2026-07-23. Same shape as `resecut` plus who deleted it.

TABLE	COLUMN	TYPE	KEY	NOTE
boletas_eliminadas	**CORR**	int	PK, auto_inc	
boletas_eliminadas	**bitacora**	int	join → bita2	the quote
boletas_eliminadas	**BOLETA**	varchar(7)		the deleted reservation
boletas_eliminadas	**JUNTO**	varchar(7)		membership number
boletas_eliminadas	**NOMBRE1**	varchar(40)		member name
boletas_eliminadas	**CVEUNI**	varchar(10)		room code
boletas_eliminadas	**cant_uni**	int		units
boletas_eliminadas	**NUMPUN**	decimal(10,1)		points
boletas_eliminadas	**impcuo**	decimal(12,2)		amount due
boletas_eliminadas	**FECDIA**	date		ledger date
boletas_eliminadas	**FECTRA**	date		transaction date
boletas_eliminadas	**NUMCON**	int		contract
boletas_eliminadas	**NUMANO**	int		contract year
boletas_eliminadas	**SECPUN**	int		sequence
boletas_eliminadas	**SEMNOC**	int		weeks/nights flag
boletas_eliminadas	**NUMSEM**	int		week number
boletas_eliminadas	**TIPDOC**	varchar(3)		document type
boletas_eliminadas	**nor_bono**	varchar(10)		points / bonus / certificate
boletas_eliminadas	**CUESTA**	varchar(2)		state
boletas_eliminadas	**DESUSO**	varchar(20)		usage note
boletas_eliminadas	**nconfir**	varchar(80)		confirmation number
boletas_eliminadas	**FE**	varchar(2)		
boletas_eliminadas	**usuario_borra**	varchar(10)		who deleted it
boletas_eliminadas	**fecha_borra**	datetime		when

comen

Where every interaction and every card charge is written, in prose.

TABLE	COLUMN	TYPE	KEY	NOTE
comen	**CORR**	int	PK, auto_inc	
comen	**bitacora**	int	join → bita2, indexed	the join
comen	**JUNTO**	varchar(10)	index	membership number
comen	**CARACTERN**	int	index	`JUNTO` as an int, prefix stripped (`914932R` → `14932`); most indexes use it
comen	**FECHA**	date	index	
comen	**HORA**	varchar(20)	index	dirty — 561 rows are `HH:MM` , 3 hold junk like `926580`
comen	**USUARIO**	varchar(20)	index	agent
comen	**GRUPO**	varchar(10)	index	department
comen	**TIPOCOM**	varchar(40)	index	note type
comen	**ESTATUS**	varchar(10)		state
comen	**COMENTA**	text		the note
comen	**COMENTA2**	text		overflow
comen	**COMENTA3**	text		overflow
comen	**COMENTA4**	text		overflow
comen	**COMENTA5**	text		overflow — a long note continues across all five

`GRUPO = 'COBROS'` with `TIPOCOM = 'RECURRENTE'` and text like `CARGO 3 CM A TC. REGISTRADA` marks a recurring card charge run by hand.

Nothing in MySQL records a charge authorization. There is no auth code, approval code, transaction id or gateway response anywhere in the schema. A `transaccion_pago` table exists with `id_transaccion` , `monto` , `ultimos_cuatro_digitos` and a `pasarela` enum of `fac` / `cyber` , but it has **0 rows**, as do `reserva_habitacion` and `reserva_evento` which reference it. So `comen` prose remains the only evidence that a charge happened or worked.

pasante

38,774 rows, 7,662 of them active. Active means `TIPCAN` is empty; any value is a cancellation reason.

TABLE	COLUMN	TYPE	KEY	NOTE
pasante	**CORR**	int	PK, auto_inc	
pasante	**JUNTO**	varchar(7)	index, **not unique**	the key used everywhere
pasante	**NUMCON**	int	index	contract
pasante	**NOMBRE1**	varchar(40)	index	primary name
pasante	**NOMBRE2**	varchar(40)	index	secondary name
pasante	**ESTATUS**	varchar(10)		`ACTIVO`
pasante	**TIPCAN**	varchar(2)	index	empty = active
pasante	**programa**	varchar(10)		`INHOPA`, `PENDIN`, …
pasante	**PUNTOS_DISPONIBLES**	int		stale — see `junto_pendiente_actualizar`
pasante	**PUNTOS_PAGADOS**	int		points paid for
pasante	**saldofav**	decimal(10,2)		credit balance
pasante	**FECCON**	date	index	contract date
pasante	**fecven**	date	index	contract expiry
pasante	**fecnac**	date		date of birth
pasante	**MAIL**	varchar(60)		email
pasante	**telcas**	varchar(20)		home phone
pasante	**telofi**	varchar(20)		office phone
pasante	**DPI**	varchar(20)		national id
pasante	**NIT**	varchar(20)		tax id
pasante	**nitnombre**	varchar(50)		billing name
pasante	**DIREC**	varchar(90)		address
pasante	**codpos**	varchar(8)		postal code

junto_pendiente_actualizar

The queue behind `actualizarCuenta(junto)`. 20,741 rows and written continuously — a member lands here when their points balance is stale, which is why `pasante.PUNTOS_DISPONIBLES` cannot be trusted for a quote.

TABLE	COLUMN	TYPE	KEY	NOTE
junto_pendiente_actualizar	**id**	int	PK, auto_inc	
junto_pendiente_actualizar	**junto**	varchar(10)		membership number
junto_pendiente_actualizar	**estatus**	enum		`pendiente` / `actualizado`
junto_pendiente_actualizar	**fecha**	datetime		queued at

emergente

Pop-up alerts shown to the agent when a member's account is opened. 13,254 rows, current.

TABLE	COLUMN	TYPE	KEY	NOTE
emergente	**corr**	int	PK, auto_inc	
emergente	**junto**	varchar(7)	index	membership number
emergente	**emergente**	text		the alert text
emergente	**activa**	varchar(8)		whether it still shows
emergente	**grupo**	varchar(20)		department
emergente	**fecha**	date		
emergente	**hora**	varchar(20)		
emergente	**usuario**	varchar(20)		who wrote it

allotment

Inventory. Already mapped in `app/mysql.py`.

TABLE	COLUMN	TYPE	KEY	NOTE
allotment	**corr**	int	PK, auto_inc	
allotment	**unidad**	varchar(10)	index	room code

TABLE	COLUMN	TYPE	KEY	NOTE
allotment	**HOTEL**	int	index	1 antigua, 2 pacifico; other values are separate properties
allotment	**entra**	date	index	check-in
allotment	**sale**	date	index	check-out
allotment	**estado**	varchar(20)	index	availability
allotment	**PTS**	varchar(15)	index	filter on `'pts'`
allotment	**JUNTO**	varchar(7)		membership number
allotment	**NOMBRE**	varchar(50)		guest
allotment	**bitacora**	int	**join → bita2, not indexed**	quote that consumed the unit
allotment	**reserva**	int		reservation
allotment	**visible_web**	varchar(2)		shown online
allotment	**MODIFICA**	date		day resolution only, NULL on ~4,900 rows — cannot be the only watermark
allotment	**CREACION**	date		created
allotment	**update_estado**	timestamp		better watermark candidate
allotment	**CONFIRMA**	varchar(60)	index	confirmation
allotment	**observa**	text		notes
allotment	**tipo**	varchar(10)	index	type
allotment	**numcon**	int		contract
allotment	**id_solires**	int		solires id

`entra` and `sale` are declared `NOT NULL`. The comment in `app/mysql.py` says they hold `0000-00-00`; on current data they do not — 67,736 rows, no zero dates, no nulls, earliest `2021-10-01`. Nothing in MySQL prevents one reappearing.

`solicitudes_allotment`

When inventory is not free, the agent raises a request and someone authorises it. Two tables: the header and one row per unit/date asked for. 5,164 and 6,709 rows, both written today.

TABLE	COLUMN	TYPE	KEY	NOTE
solicitudes_allotment	**id**	int	PK, auto_inc	
solicitudes_allotment	**hotel**	int	index	hotel
solicitudes_allotment	**unidad**	int	index	unit type
solicitudes_allotment	**cantidad**	int		units wanted
solicitudes_allotment	**fecha_entrada**	date		check-in
solicitudes_allotment	**fecha_salida**	date		check-out
solicitudes_allotment	**estatus**	enum		only `si_hay`, `denegada`, `pendiente` occur
solicitudes_allotment	**status_cp**	enum		secondary state, set on 10%
solicitudes_allotment	**usuario_sol**	varchar(50)	index	who asked
solicitudes_allotment	**usuario_auto**	varchar(50)	index	who authorised
solicitudes_allotment	**fecha_respuesta**	datetime		answered at
solicitudes_allotment	**contrato**	varchar(100)		contract, on 43%
solicitudes_allotment	**porcentaje**	decimal(10,2)		discount, on 11%
solicitudes_allotment	**nombre_reserva**	varchar(200)		guest, on 5%
solicitudes_allotment	**comentario**	text		
solicitudes_allotment	**created_at**	timestamp		
solicitudes_allotment	**updated_at**	timestamp		auto-updates
detalle_soli_allotment	**id**	int	PK, auto_inc	
detalle_soli_allotment	**id_solicitud**	int	index → solicitudes_allotment.id	the header
detalle_soli_allotment	**unidad**	varchar(10)		room code — text here, `int` on the header
detalle_soli_allotment	**entra**	date		check-in
detalle_soli_allotment	**sale**	date		check-out
detalle_soli_allotment	**estatus**	enum		per-line state
detalle_soli_allotment	**porcentaje**	int		discount, on 13%
detalle_soli_allotment	**comentario**	text		on 19%
detalle_soli_allotment	**created_at**	timestamp		
detalle_soli_allotment	**updated_at**	timestamp		auto-updates

`unidad` is an `int` on the header and a `varchar(10)` room code on the detail — they are not the same kind of value.

bono_otorgado

Bonus points granted to a member, 31,232 rows, current.

TABLE	COLUMN	TYPE	KEY	NOTE
bono_otorgado	**CORR**	int	PK, auto_inc	
bono_otorgado	**JUNTO**	varchar(7)	index	membership number
bono_otorgado	**NUMPUN**	decimal(7,2)		points granted
bono_otorgado	**tipo**	varchar(10)		bonus type
bono_otorgado	**fecha**	date		granted on
bono_otorgado	**observa**	varchar(100)		reason

saldofav

The credit-balance ledger, 15,422 rows, current. `pasante.saldofav` is the running total; this is the movement detail, and it links back to both the quote and the reservation.

TABLE	COLUMN	TYPE	KEY	NOTE
saldofav	**CORR**	int	PK, auto_inc	
saldofav	**JUNTO**	varchar(7)	index	membership number
saldofav	**FECHA**	datetime		when
saldofav	**CREDITO**	decimal(12,2)		amount in
saldofav	**DEBITO**	decimal(12,2)		amount out
saldofav	**DESCRIP**	varchar(250)		description
saldofav	**motivo**	varchar(15)		reason code, on 48%
saldofav	**bitacora**	int	index → **bita2**	the quote, on 46%
saldofav	**boleta**	int	→ **bita_reserva**	the reservation, on 35%
saldofav	**USUARIO**	varchar(30)		agent

Every row has exactly one of `CREDITO` or `DEBITO`, never both.

credit_cards

The card-on-file store. 39,763 cards covering 38,459 distinct members, written today, and 39,760 of the rows match a `pasante` row. This is the live vault — the card columns on `bita2` and `pasante` are the older duplicates.

TABLE	COLUMN	TYPE	KEY	NOTE
credit_cards	**corr**	int	PK, auto_inc	
credit_cards	**junto**	varchar(33)	index	membership number — `varchar(33)` where every other table uses 7
credit_cards	**numero**	varchar(25)		**card number in the clear**, on 93%
credit_cards	**nombre**	varchar(100)		cardholder name
credit_cards	**vence_mes**	int		expiry month, on 32%
credit_cards	**vence_ano**	int		expiry year, on 32%
credit_cards	**forma_pago**	varchar(100)	index	`CRÉDITO` / `DÉBITO` / `DEPOSITO`
credit_cards	**network**	varchar(100)	index	card network, on 47%
credit_cards	**issuing_bank**	varchar(100)	index	issuer, on 90%
credit_cards	**es_principal**	tinyint		the member's default card
credit_cards	**es_oficial**	tinyint	index	verified card
credit_cards	**created_at**	datetime		
credit_cards	**updated_at**	datetime		auto-updates
credit_cards	**api_bin**	varchar(20)		BIN, from the lookup API, on 62%
credit_cards	**api_marca**	varchar(60)		brand, on 63%
credit_cards	**api_tipo**	varchar(60)		credit/debit, on 63%
credit_cards	**api_esquema**	varchar(60)		scheme, on 62%
credit_cards	**api_pais**	varchar(60)		country, on 62%
credit_cards	**api_banco**	varchar(80)	index	issuer name, on 91%
credit_cards	**api_actualizado**	datetime	index	last BIN refresh, on 13%

The `api_*` columns are BIN-lookup results cached on the row. `numero` holds the full number, so `api_bin` is derivable and the card itself is what needs to leave.

credit_cards_log

Every change to a `credit_cards` row, as before/after JSON. 131,040 rows — 31,567 inserts and 99,473 updates — written today.

TABLE	COLUMN	TYPE	KEY	NOTE
credit_cards_log	**id**	int	PK, auto_inc	
credit_cards_log	**credit_card_corr**	int	→ credit_cards.corr	which card
credit_cards_log	**accion**	enum		`INSERT` / `UPDATE` / `DELETE`
credit_cards_log	**usuario**	varchar(100)		who changed it
credit_cards_log	**fecha**	datetime		when
credit_cards_log	**data_anterior**	json		the row before
credit_cards_log	**data_nueva**	json		the row after

This logs edits to the card record, **not charges**. The JSON carries `numero`, so every historical card number is retained here even after the card row is corrected or deleted.

Card lookups

TABLE	COLUMN	TYPE	KEY	NOTE
forma_pago	**corr**	int	PK, auto_inc	
forma_pago	**descripcion**	varchar(100)		`CRÉDITO` `DÉBITO` `DEPOSITO`
tipo_tarjeta	**corr**	int	PK, auto_inc	
tipo_tarjeta	**descripcion**	varchar(100)	unique	same three values as `forma_pago`
network	**id**	int	PK, auto_inc	
network	**descripcion**	varchar(100)	unique	`VISA` `MASTERCARD` `AMEX` `DINERS` `DISCOVER` `JCB` `UNIONPAY` `DEPOSITO`
type_acquiring	**id**	int	PK, auto_inc	
type_acquiring	**descripcion**	varchar(100)	unique	`PLATINO` `ORO` `CLASICA`
issuing_bank	**id**	int	PK, auto_inc	

TABLE	COLUMN	TYPE	KEY	NOTE
issuing_bank	**descripcion**	varchar(100)	unique	2,374 issuer names
bin_list_tarjetas	**corr**	int	PK, auto_inc	
bin_list_tarjetas	**bin**	int	index	177,405 BIN ranges
bin_list_tarjetas	**brand**	varchar(25)		
bin_list_tarjetas	**type**	varchar(11)		credit / debit
bin_list_tarjetas	**category**	varchar(34)		
bin_list_tarjetas	**issuer**	varchar(96)		
bin_list_tarjetas	**countryname**	varchar(13)		
bin_list_tarjetas	**isocode3**	varchar(3)		
bin_cache	**bin**	int	PK	live API results, 340 rows
bin_cache	**scheme**	varchar(30)		
bin_cache	**type**	varchar(30)		
bin_cache	**brand**	varchar(60)		
bin_cache	**bank**	varchar(80)		
bin_cache	**country**	varchar(60)		
bin_cache	**updated_at**	datetime		

`bin_list_tarjetas` is the bulk BIN table; `bin_cache` is the per-BIN cache of the live API that fills `credit_cards.api_*`.

Lookup tables

TABLE	COLUMN	TYPE	KEY	NOTE
tipo_sol	**id**	int	PK	feeds the Medio dropdown → `bita2.TIPOSOL`
tipo_sol	**name**	varchar(255)		the stored value
tipo_sol	**title**	varchar(255)		the label shown
tipo_reserva	**id**	int	PK	feeds Tipo Reserva → `bita2.ASUNTO`
tipo_reserva	**name**	varchar(255)		the stored value
tipo_reserva	**title**	varchar(255)		the label shown

TABLE	COLUMN	TYPE	KEY	NOTE
tipo_unid	**id**	int	PK, auto_inc	room types
tipo_unid	**unidad**	varchar(10)	index	room code
tipo_unid	**hotel**	int	index	hotel
tipo_unid	**nombre**	varchar(50)		display name
tipo_unid	**descripcion**	text		description
tipo_unid	**info**	text		extra info
tipo_unid	**max_unid**	int		max units
tipo_unid	**adultos**	int		max adults
tipo_unid	**ninos**	int		max children
tipo_unid	**color**	varchar(50)		UI colour
tipo_unid	**img_portada**	mediumblob		cover image stored in the row
combinaciones	**id**	bigint unsigned	PK, auto_inc	party size to room mapping
combinaciones	**hotel**	int		hotel
combinaciones	**adultos**	tinyint unsigned		adults
combinaciones	**ninos**	tinyint unsigned		children
combinaciones	**total**	tinyint unsigned		party size
combinaciones	**prioridad**	smallint unsigned		ordering
combinaciones	**activo**	tinyint(1)		on/off
combinaciones	**created_at**	timestamp		
combinaciones	**updated_at**	timestamp		
destino	**CORR**	int	PK, auto_inc	season and points rates
destino	**CVELUG**	int	index	place code
destino	**DESTINO**	varchar(20)		season code
destino	**destino_res**	varchar(10)		season code as used on reservations
destino	**CVEUNI**	varchar(10)	index	room code
destino	**CVETEM**	varchar(1)		season type
destino	**CVEEST**	varchar(1)		state
destino	**NUMPUN**	decimal(6,1)	index	points rate

TABLE	COLUMN	TYPE	KEY	NOTE
destino	**IMPCTA**	decimal(12,2)		amount
destino	**SECPUN**	int		sequence
destino	**ANOCTA**	int	index	year
destino	**DESUSO**	varchar(20)		usage
destino	**FINENTRE**	varchar(6)		weekend / weekday
destino	**LOCALINTER**	varchar(5)		local / international
destino	**TIPO**	varchar(10)	index	type
destino	**NOM_COMPLETO**	varchar(50)		full name
destino	**lugaruso**	varchar(50)		place
destino	**BAJA**	varchar(1)	index	inactive flag

`tipo_sol` holds twelve channels including `mercadeo`, which the form offers but no `bita2` row has ever stored. `tipo_reserva` holds exactly the seven `ASUNTO` values plus a blank `Seleccione...` placeholder.

Coded values

Every value below actually occurs in the data.

TABLE.COLUMN	VALUES
bita2.RESULTADO	**confirmada Informacion No hay espacios Lista de Espera**
bita2.ESTATUS	**CERRADO pendiente ANULADO**
bita2.TIPOSOL	**whatsapp telefono presencial web correo ventas empleado inhouse otros tmk webchat**
bita2.ASUNTO	**PUNTOS BONO CERTIFICADO PUNTOSBONO CERTIPUNTOS INTERCAMBIO MIXTA**
bita2.TIPO_RESERVA	**PUNTOS BONO CERTIFICADO**
bita2.HOTEL1	**1** antigua · **2** pacifico
bita_reserva.ESTATUS	**UTILIZADA CANCELADO**
resecut.TIPDOC	**VI VE EX EQ CT EM RE UP**
resecut.nor_bono	**PUNTOS BONO CERTIFICADO**
resecut.CUESTA	**U** used · **C** cancelled · **R** returned

TABLE.COLUMN	VALUES
comen.GRUPO	**COBROS SERVICIO POSVENTA CREDITOS**
comen.TIPOCOM	**RECURRENTE NORMAL CR-NORMAL SOLEIL PACIFICO CR-NOTIF COBROS SOLEIL LA ANTIGUA AS-RESERVA**
pasante.TIPCAN	empty = active · **CV 19 AM 02 16 UP PS**
solicitudes_allotment.estatus	**si_hay denegada pendiente**
tarifa_hotel3.temporada	**ENTRE SEMANA FIN DE SEMANA SUPER ALTA PROMO ENTRE FIN**
credit_cards_log.accion	**INSERT UPDATE DELETE**

`bita2.RESULTADO` also holds blanks and two stray values `1` and `2`. `ANULADO` is only 68 rows, all pre-2025.

Legacy card columns

Card numbers live in the clear in three generations of table, all still present.

TABLE	COLUMN	TYPE	NOTE
credit_cards	**numero**	varchar(25)	**current** — 39,763 rows, written today
credit_cards_log	**data_nueva / data_anterior**	json	every historical value of the above
pasante	**TARNUMERO**	varchar(30)	card on file — 5,897 of 7,662 active members
pasante	**TARNOMBRE**	varchar(100)	cardholder name, and it is **indexed**
pasante	**tarmes / tarano**	int	expiry
bita2	**TARJETA**	varchar(20)	17,305 of 30,174 parse as a 13–19 digit PAN
bita2	**TARJETAMES / TARJETAANO**	varchar	expiry
bita2	**TARJETAEMI**	varchar(20)	issuer
bita2	**COMENTARIO**	varchar(25)	nominally a code field; agents type card numbers into it
bita2	**ASUNTO2**	varchar(40)	same — card numbers, NITs and emails as free text
credit_card	**numero**	varchar(25)	77 rows, superseded by **credit_cards**
credit_card_manif	**numero**	varchar(25)	8,366 rows, sales-manifest copy

Keep a processor token plus `last4`, `brand`, `exp_month`, `exp_year` instead, and record transaction id, auth code, amount, status and timestamp — none of which exists today. `app/bac.py` already pings `staging.ptranz.com`.

About ten backup copies of `bita2` and `bita_reserva` also live in the schema (`bita2_bk_26_01_2026`, `bita2_rescue`, `bita2_solo_ingreso`, …) carrying the same card columns.